

Gestione dei file

Gestione dei file

Come leggere e scrivere file con Python — per salvare e caricare dati.

Perché lavorare con i file?

Quando un programma si chiude, tutto quello che era in memoria scompare. Se vuoi che i dati sopravvivano tra una esecuzione e l'altra (come i punteggi di un gioco, o una lista di nomi), devi **salvarli su file**.

Aprire un file

Per lavorare con un file, lo "apri" con la funzione `open()`. Devi specificare il nome del file e la **modalità** (cosa vuoi fare):

Modalità	Cosa fa
"r"	Legge il file (deve esistere)
"w"	Scriva nel file (lo crea se non esiste, lo svuota se esiste)
"a"	Aggiunge alla fine del file senza cancellare il contenuto

Il modo corretto: il blocco `with`

Usa sempre il costrutto `with` per aprire i file. Garantisce che il file venga chiuso correttamente anche se c'è un errore:

```
with open("esempio.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
# Il file viene chiuso automaticamente quando esci dal blocco with
```

Leggere un file

Leggere tutto il contenuto in una volta:

```
with open("esempio.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
```

Leggere riga per riga (utile per file grandi):

```
with open("esempio.txt", "r") as file:
    for riga in file:
        print(riga.strip()) # strip() rimuove il carattere di a capo alla
fine
```

Ottenere tutte le righe come lista:

```
with open("esempio.txt", "r") as file:
    righe = file.readlines() # lista di stringhe, una per riga

print(righe)
```

Scrivere su un file

Creare un nuovo file (o sovrascriverlo):

```
with open("output.txt", "w") as file:
    file.write("Prima riga\n")
    file.write("Seconda riga\n")
    file.write("Terza riga\n")
```

:::caution La modalità `"w"` cancella tutto il contenuto precedente del file. Se vuoi aggiungere senza cancellare, usa `"a"`. :::

Aggiungere righe a un file esistente:

```
with open("output.txt", "a") as file:
    file.write("Questa riga viene aggiunta alla fine\n")
```

Gestire il caso in cui il file non esiste

Se provi ad aprire un file che non esiste in modalità `"r"`, Python dà un errore. Usa `try/except` per gestirlo:

```
try:
    with open("dati.txt", "r") as file:
        contenuto = file.read()
        print(contenuto)
except FileNotFoundError:
    print("Il file non esiste ancora.")
```

Esempio pratico: salvare e caricare una lista

```
def salva_studenti(studenti, nome_file):
    """Salva la lista degli studenti su file."""
    with open(nome_file, "w") as file:
        for nome, voto in studenti:
            file.write(f"{nome},{voto}\n")

def carica_studenti(nome_file):
    """Carica la lista degli studenti dal file."""
    studenti = []
    try:
        with open(nome_file, "r") as file:
            for riga in file:
                parti = riga.strip().split(",")
                if len(parti) == 2:
                    nome = parti[0]
                    voto = float(parti[1])
                    studenti.append((nome, voto))
    except FileNotFoundError:
        print("File non trovato, parto con lista vuota.")
    return studenti

# Salva i dati su file
studenti = [("Alice", 8.5), ("Bob", 7.0), ("Carlo", 9.2)]
salva_studenti(studenti, "studenti.csv")

# Ricarica i dati dal file
caricati = carica_studenti("studenti.csv")
for nome, voto in caricati:
    print(f"{nome}: {voto}")
```

Il file `studenti.csv` conterrà:

```
Alice,8.5
Bob,7.0
Carlo,9.2
```